MySQL Stored Procedure Parameters

Summary: in this tutorial, you will learn how to create stored procedures with parameters, including `IN`, `OUT`, and `INTOUT` parameters.

Introduction to MySQL stored procedure parameters

Typically, stored procedures have parameters, making them more useful and reusable.

A parameter in a stored procedure has one of three modes: `IN`, `OUT`, or `INOUT`.

IN parameters

`IN` is the default mode. When defining an `IN` parameter in a stored procedure, the calling program must pass an argument to the stored procedure.

Additionally, the value of an `IN` parameter is protected. This means that even if you change the value of the `IN` parameter inside the stored procedure, its original value remains unchanged after the stored procedure ends. In other words, the stored procedure works only on the copy of the `IN` parameter.

OUT parameters

The value of an `OUT` parameter can be modified within the stored procedure, and its updated value is then passed back to the calling program.

> Note that stored procedures cannot access the initial value of the `OUT` parameter when they begin.

INOUT parameters

An `INOUT` parameter is a combination of `IN` and `OUT` parameters. This means that the calling program may pass the argument, and the stored procedure can modify the `INOUT` parameter and pass the new value back to the calling program.

Defining a parameter

Here is the basic syntax for defining a parameter in stored procedures:

```
[IN | OUT | INOUT] parameter_name datatype[(length)]
```

In this syntax,

- First, specify the parameter mode, which can be `IN` , `OUT` or `INOUT` depending on the purpose of the parameter in the stored procedure.
- Second, provide the name of the parameter. The parameter name must follow the naming rules of the column name in MySQL.
- Third, define the data type and maximum length of the parameter.

MySQL stored procedure parameter examples

Let's explore some examples of using stored procedure parameters.

The IN parameter example

The following example creates a stored procedure that finds all offices that are located in a country specified by the input parameter `countryName` :

```
DELIMITER //
```

```
CREATE PROCEDURE GetOfficeByCountry(
        IN countryName VARCHAR(255)
)
BEGIN
        SELECT *
        FROM offices
        WHERE country = countryName;
END //

DELIMITER ;
```

In this example, the `countryName` is the `IN` parameter of the stored procedure.

Suppose that you want to find offices located in the USA, you need to pass an argument (`USA`) to the stored procedure as shown in the following query:

```
CALL GetOfficeByCountry('USA');
```

	officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
▶	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA

To find offices in `France` , you pass the literal string `France` to the `GetOfficeByCountry` stored procedure as follows:

```
CALL GetOfficeByCountry('France')
```

Because the `countryName` is the `IN` parameter, you must pass an argument. If you don't do so, you'll get an error:

```
CALL GetOfficeByCountry();
```

Here's the error:

```
Error Code: 1318. Incorrect number of arguments for PROCEDURE classicmodels.GetOfficeByCou
```

The OUT parameter example

The following defines a stored procedure that returns the number of orders based on their order status.

```
DELIMITER $$

CREATE PROCEDURE GetOrderCountByStatus (
        IN  orderStatus VARCHAR(25),
        OUT total INT
)
BEGIN
        SELECT COUNT(orderNumber)
        INTO total
        FROM orders
        WHERE status = orderStatus;
END$$

DELIMITER ;
```

The stored procedure `GetOrderCountByStatus()` has two parameters:

- The `orderStatus` is the `IN` parameter specifies the status of orders to return.
- The `total` is the `OUT` parameter that stores the number of orders in a specific status.

To find the number of orders that already shipped, you call `GetOrderCountByStatus` and pass the order status as of `Shipped` , and also pass a session variable (`@total`) to receive the return value.

```
CALL GetOrderCountByStatus('Shipped',@total);
SELECT @total;
```

Output:

```
+--------+
| @total |
```

```
+--------+
|    303 |
+--------+
1 row in set (0.00 sec)
```

To get the number of orders that are in process, you call the stored procedure `GetOrderCountByStatus` as follows:

```
CALL GetOrderCountByStatus('In Process',@total);
SELECT @total AS total_in_process;
```

Output:

```
+------------------+
| total_in_process |
+------------------+
|                6 |
+------------------+
1 row in set (0.00 sec)
```

The INOUT parameter example

The following example demonstrates how to use an `INOUT` parameter in a stored procedure:

```
DELIMITER $$

CREATE PROCEDURE SetCounter(
        INOUT counter INT,
    IN inc INT
)
BEGIN
        SET counter = counter + inc;
END$$

DELIMITER ;
```

In this example, the stored procedure `SetCounter()` accepts one `INOUT` parameter (`counter`) and one `IN` parameter (`inc`). It increases the counter (`counter`) by the value specified by

the `inc` parameter.

These statements illustrate how to call the `SetSounter` stored procedure:

```
SET @counter = 1;
CALL SetCounter(@counter,1); -- 2
CALL SetCounter(@counter,1); -- 3
CALL SetCounter(@counter,5); -- 8
SELECT @counter; -- 8
```

Here is the output:

In this tutorial, you have learned how to create stored procedures with parameters including `IN`, `OUT`, and `INOUT` parameters.

PREVIOUSLY
MySQL CREATE PROCEDURE

UP NEXT
MySQL DROP PROCEDURE

Search ...

STORED PROCEDURE BASICS

CONDITIONAL STATEMENTS

LOOPS

ERROR HANDLING

Show Errors

Declare ... Condition Statement

DECLARE ... HANDLER Statement

RESIGNAL Statement

CURSORS

MySQL Cursor

SECURITY

Stored Object Access Control

ABOUT MYSQL TUTORIAL WEBSITE

MySQLTutorial.org helps you master MySQL quickly, easily, and with enjoyment. Our tutorials make learning MySQL a breeze.

All MySQL tutorials are clear, practical and easy-to-follow. More About Us

LATEST TUTORIALS

MySQL Port

MySQL Commands

innodb_dedicated_server: Configure InnoDB Dedicated Server

innodb_flush_method: Configure InnoDB Flush Method

innodb_log_buffer_size: Configure InnoDB Log Buffer Size

innodb_buffer_pool_chunk_size: Configure Buffer Pool Chunk Size

innodb_buffer_pool_instances:

SITE LINKS

Donation 💔

Contact Us

About

Privacy Policy

OTHERS

MySQL Cheat Sheet

MySQL Resources

MySQL Books

Configuring Multiple Buffer Pool Instances for Improved Concurrency in MySQL

innodb_buffer_pool_size: Configure InnoDB Buffer Pool Size

MySQL InnoDB Architecture

How to Kill a Process in MySQL